TECHNICAL REFERENCE GUIDE

Qwik Guide: CNNs & RNNs

Convolutional Neural Networks · Recurrent Neural Networks · LSTMs

A beginner-friendly companion for the AI Foundations Series.

Plain-language definitions, step-by-step processes, and diagrams — a partner volume to the Unsupervised Learning guide.

Some problems don't fit an ordinary neural network. Images have structure in space; language and time series have structure in *order*. This guide explains the two specialized networks built for those jobs — **convolutional** networks for images and **recurrent** networks for sequences — defining every term as it appears.

Convolutional Networks, From the Ground Up

A regular neural network looks at an **entire input at once**. For an image, that's a problem: the same letter drawn in a different spot or at a different angle becomes a completely different input, so the network has to re-learn it from scratch. A **convolutional neural network (CNN)** fixes this by looking at **small patches** of the image at a time and reusing the same detector everywhere.

> **TIP** Read this after the basic neural-network section. The single biggest idea here: a CNN scans the image in small overlapping windows instead of swallowing the whole thing at once — that's what makes it good at images and audio.

1 Regular Network vs. CNN

In a regular network every pixel feeds one giant input layer, so position matters and the network is easily confused by shifts. In a CNN, a small **filter** slides across the image, and the *same* weights are applied at every position — so a feature learned in one corner is recognized everywhere.

Regular network → sees the whole image at once **Convolutional network → scans small patches**

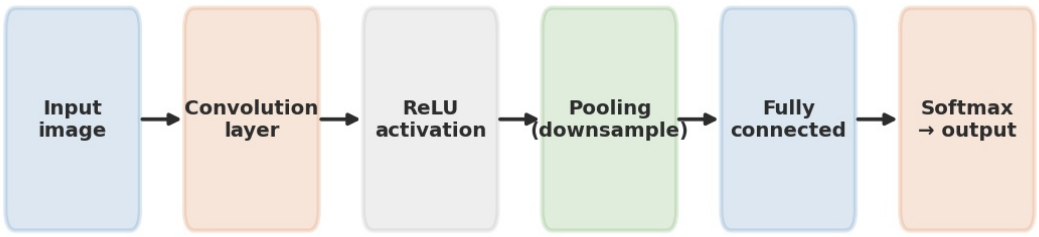
Left: a regular network treats the whole image as one input, so the same character in a new position looks brand new. Right: a CNN slides a small filter across the image, reusing the same detector at every spot.

Key vocabulary

- **Convolutional layer** — a layer that is *not* fully connected and uses **weight-sharing units**: many copies of one identical neuron. This gives a big, powerful network while keeping the number of parameters small.
- **Overlapping patches** — a filter is applied to small, overlapping patches of the input: it grabs one patch, feeds it to a neuron, then moves over (usually by one pixel) and repeats until the whole image is covered.
- **Weight sharing** — because the copied units are identical, they share the same weights, so they all detect the same feature.
- **Common uses** — image processing, character recognition, image tagging, audio processing, and game playing (e.g. Go and classic Atari games).

2 CNN Architecture

A CNN stacks a few kinds of layers in order. Data flows from the raw image through feature-detecting layers, gets shrunk, and ends as a classification.



The typical CNN pipeline: convolution finds features, ReLU keeps the useful ones, pooling shrinks the data, and the fully-connected and softmax layers turn it into a labelled output.

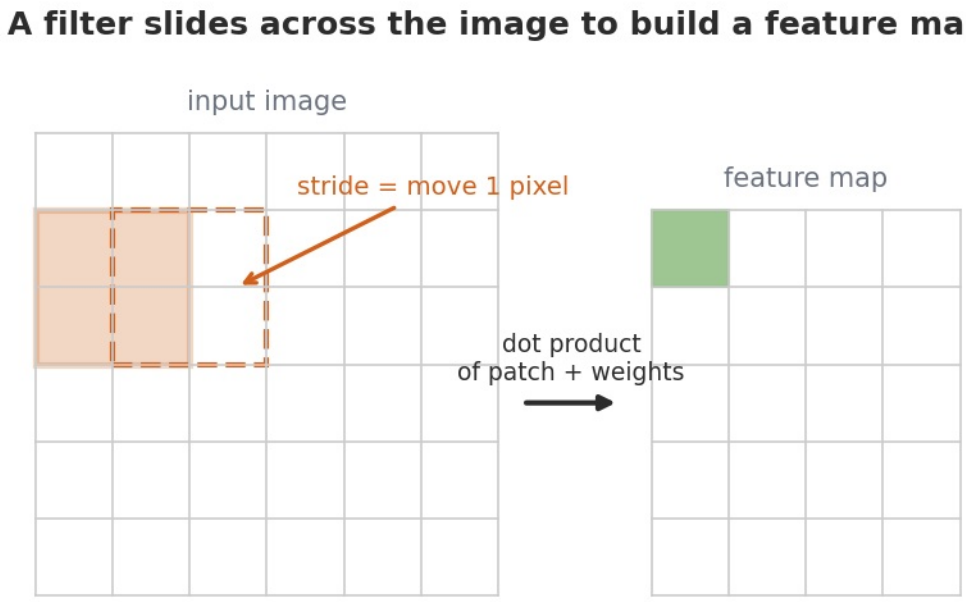
Architecture terms

- **Convolutional layer** — extracts different features from each part of the image.
- **Filter** — a set of weights applied to a section of the image. A 5×5 filter covers a 5-pixel by 5-pixel patch; a 5×5×3 filter adds three colour channels.
- **Feature map** — the new image created when a filter is applied across the input.
- **Stride** — how far the filter moves each step. Stride size is *how much to move*; a stride of 1 shifts the filter one pixel.
- **ReLU, fully-connected, softmax** — the activation layer, the ordinary classifier layer(s), and the final layer that normalises the numbers into class probabilities.

EXAMPLE LeNet-5, one of the first CNNs, recognized characters from 32×32 images using 7 layers: convolutional layers with 5×5 filters, pooling layers that downsample to smaller feature maps, a fully-connected layer, and an output layer — ending in a softmax that reads off the letter.

3 How a Convolution Layer Works

To apply a filter, the network takes the **dot product** of the image patch and the filter weights, producing one number. Slide the filter by the stride, repeat, and the collected outputs form the feature map. To implement the sliding, the same unit is **copied** many times — each copy shares the same weights and outputs one piece of the feature map.

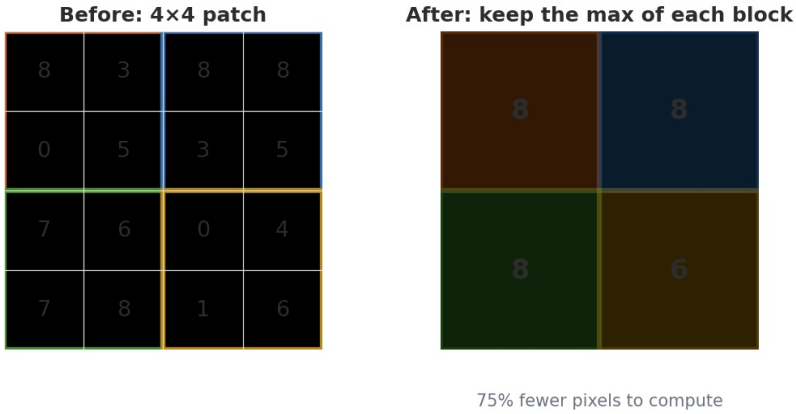


The filter (orange) covers a patch, produces one value by a dot product, then strides one pixel and repeats — filling in the feature map (green) one cell at a time.

THE KEY IDEA Because the same filter (the same weights) is reused at every position, a CNN learns each feature **once** and can then spot it anywhere in the image — that's the payoff of weight sharing.

4 Pooling Layers

Pooling **shrinks each feature map** by downsampling — keeping fewer values from every region. This cuts the number of **activations** that later layers must handle (and, in turn, the parameters those layers need). Pooling layers sit **between** convolutional layers. Their advantages: fewer computations, less memory, and less **overfitting** (the statistical trap where noise hides the real pattern in the data).



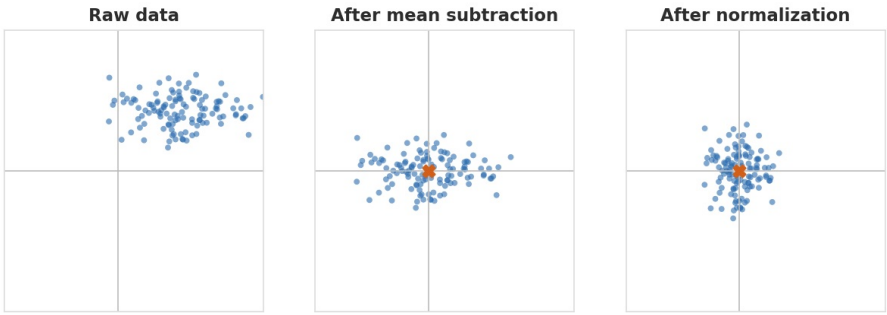
2×2 max pooling: for each 2×2 block, keep only the largest value. A 32×32 image becomes 16×16 — 75% fewer activations to compute.

- **Max pooling** — returns the maximum pixel in each block.
- **Average pooling** — averages the pixels being downsampled.
- **Global average pooling** — averages whole feature maps so they can be combined; sometimes used to replace a final fully-connected layer.

5 Training Considerations

CNNs need training, and a few steps differ from ordinary networks.

Preprocessing



Mean subtraction re-centers the data on the origin; normalization then rescales every feature to the same spread.

- **Mean subtraction** — subtract the mean from each example so the data is centered on the origin.
- **Normalization** — divide by the standard deviation so every feature is on the same scale.
- **Zero padding** — add a border of zeros (e.g. pixels around an image).

> **TIP** If you split into training and test sets, do the preprocessing **after** the split — and apply the same steps to anything you later want to predict.

Weights and training

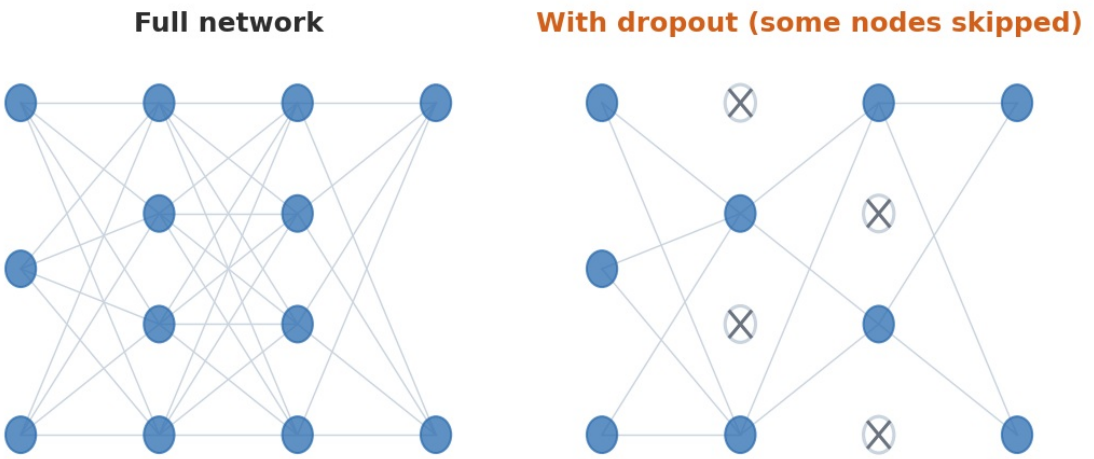
- **Don't initialize all weights to zero** — they'd all update identically. Use small random numbers, then divide by the square root of the number of inputs to reduce variance.
- **Backpropagation with gradient descent** trains the network. Fully-connected and convolutional layers train like normal layers; because weights are shared, their gradients are shared too.
- **Pooling layers have no weights**, so they need no training.

6 Regularization

Regularization reduces overfitting by penalizing models that are prone to it. Its underlying *mechanism* is **weight decay**: large weights are pushed toward zero over time. A one-off outlier that spikes a weight fades away, while a weight that *should* be large is held up by many supporting examples. The methods below — L1, L2, max-norm, dropout, and early stopping — are different ways of applying this idea; weight decay itself is not a separate method in that list.

Common methods

- **L1 regularization** — penalty of $\lambda \cdot |w|$. Encourages weights near zero, making noise less relevant.
- **L2 regularization** — penalty of $\lambda \cdot w^2/2$. Discourages relying on a small subset of inputs; it weighs more inputs equally. Generally L2 performs better.
- **Max-norm constraint** — caps the magnitude of a unit's weights (the Pythagorean length of the weight vector), enforced during updates.
- **Dropout** — randomly skips some activations during training (a common setting is p = 0.5). It should **not** be applied when making predictions.



Dropout randomly switches off nodes during training, so no node leans too heavily on any other — an efficient way of averaging many models.

Regularization methods aimed at CNNs

- **Modify the dataset** — more training data often improves an overfitting model.
- **Dropout layer** — randomly drops nodes together with their inputs and outputs.
- **Early stopping** — halts iterative training (like gradient descent) once validation performance stops improving, before the model begins to overfit. The number of training steps effectively becomes a tuned hyperparameter.
- **Weight penalties (L1 / L2)** — keep weights small on the assumption that smaller weights mean a simpler, less overfit model.

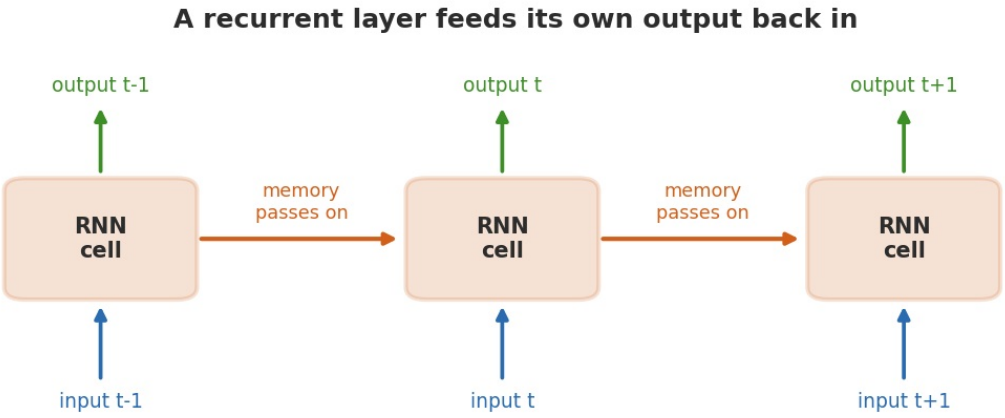
> **WATCH OUT** "Weight decay" is the underlying *mechanism* regularization uses — not a separate method to list alongside dropout, max-norm, and L2.

Recurrent Networks, From the Ground Up

Ordinary networks have **no memory** — they forget after every step. That makes them poor at **sequential** or time-based data, where the past shapes the present: time series, video, and language. A **recurrent neural network (RNN)** feeds its own output back into itself, so information carries forward.

7 What Is a Recurrent Layer?

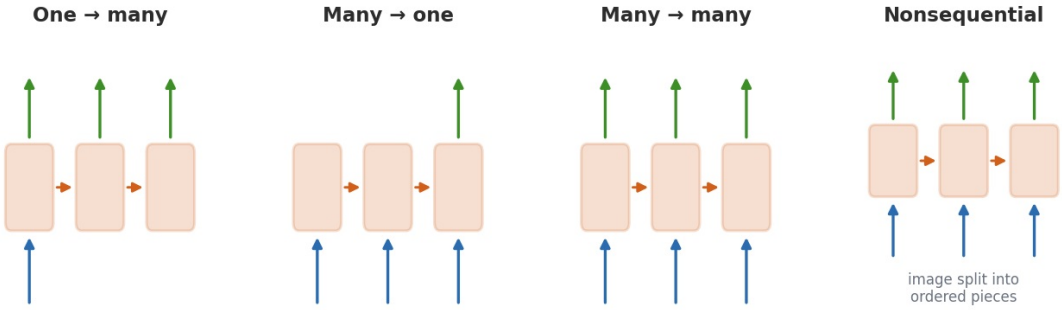
A **recurrent layer** is a layer that feeds output into itself **sequentially**: the output of the last step is used in the next one, forming a chain of identical copies that pass information along. Each step's output depends on what came before, giving the network a working form of memory.



Each step takes a new input and the previous step's output, so information is retained down the chain — a simple form of memory.

THE LIMITATION A plain RNN passes forward only the previous step's output, so in practice it struggles to hold on to information across many steps — its ability to retain **long-range dependencies** is limited. That's the gap LSTMs close (next page).

8 Types of RNN



The four arrangements below cover most RNN use cases.

- **One input → many outputs** — e.g. describing a single input across several outputs.
- **Many inputs → one output** — e.g. sentiment analysis, where a whole post is read and one verdict (happy, sad, angry) is produced.
- **Many inputs → many outputs** — can be *unsynced* (machine translation, where word order differs) or

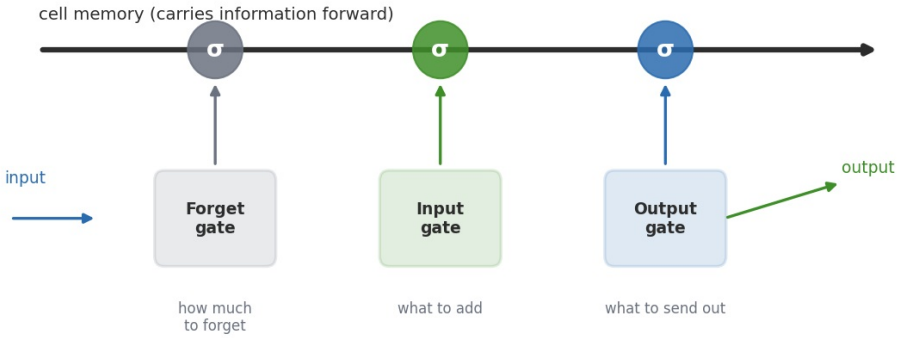
9 Long Short-Term Memory (LSTM)

LSTM networks are recurrent networks that *can* learn relationships over longer time spans. The fix for the plain RNN's limited grip on long-range dependencies is to add a **memory cell**. Weights learn what to **forget**, what to **remember**, and how to **use** the memory.

Reading the LSTM diagram

- **Cell** — represents the memory that runs through the unit.
- **× and +** — multiply and add gates.
- **Sigmoid (σ)** — a logistic unit; returns a number between 0 and 1, used as a *percentage* of how much to keep.
- **Tanh** — a unit that extracts information to add or read out.
- **t** — the current time or iteration.

Inside an LSTM cell: a memory line with three gates



An LSTM runs a memory line through three sigmoid gates: forget (how much to drop), input (what to add), and output (what to pass on).

How an LSTM remembers — three steps

- **Step 1 — how much to forget.** A sigmoid returns a value between 0 and 1, the percentage of the old memory to keep.
- **Step 2 — what to add.** A tanh proposes new information; a sigmoid decides what percentage of it to add; the result is written to the cell.
- **Step 3 — build the output.** A tanh reads from the cell, a sigmoid decides how much to send, and the output passes to the next step.

THE GATE FORMULAS Both gate types share the same shape — each looks at the current input X_t and the previous output h_{t-1}:

Sigmoid gate: $\sigma(W \cdot [X_t, h_{t-1}] + b)$

Tanh gate: $tanh(W \cdot [X_t, h_{t-1}] + b)$

10 RNNs & LSTM for Language Modeling

A language model fits a **probabilistic model** that assigns probabilities to sentences by prediction: *given the previous words, what comes next?* An LSTM layer processes one word at a time and predicts the probability of each next word. In the classic TensorFlow example, the model trains on the PTB dataset (10,000 words) to predict the next word in a sentence.

★ Concept Check

These are the exact points students most often trip on — drawn from common exam-style questions on this material. Read each as "the trap, then the truth."

Which is *not* a type of RNN?
One-to-many, many-to-one, many-to-many, and nonsequential are all real. **"One output to many inputs" is not** — inputs feed in, outputs come out, not the reverse.

What does a convolution layer do?
It **extracts different features from each part of an image**. (Assigning weights to an area is the *filter*'s job.)

What is a filter?
A **set of weights applied to a section of an image** — not the resulting image, and not the amount to move.

In an LSTM, what is a logistic unit?
The **sigmoid**. (The cell is memory; tanh is a different unit.)

Which is *not* a regularization method?
Weight decay — it's the *mechanism* behind regularization, not a method beside dropout, max-norm, and L2.

What does pooling reduce?
The **number of parameters** in the network — by shrinking feature maps and the activations they carry (which also cuts computation and overfitting).

Which is *not* a quality of a convolutional layer?
Three things are true of a convolutional layer: it isn't fully connected, it processes small parts of the image, and it shares weights. The false one is **"requires no prior training"** — a convolutional layer **is trained**, like the rest of the network.

What is a recurrent layer?
A layer that **feeds output into itself sequentially**.

What happens in preprocessing?
Mean subtraction (center the data), then normalization and zero padding.

What does convolution input refer to?
A filter applied to small, **overlapping patches** of the input — not the whole image at once.

Which method stops training once validation performance stops improving?
Early stopping — the number of steps becomes a tuned hyperparameter.

Why is connecting units recurrently limited?
Because a plain RNN passes forward mainly the **last step's output**, so it struggles to retain **long-range dependencies**.

★ Quick Reference

Concept	Type	Remember this
Convolutional layer	**CNN**	Not fully connected; shares weights; extracts features from each part of the image
Filter	**CNN**	A set of weights applied to a patch; slides across via the stride
Feature map	**CNN**	The new image built when a filter is applied across the input
Stride	**CNN**	How far the filter moves each step (stride 1 = one pixel)
Pooling	**CNN**	Downsamples to cut parameters, computation, and overfitting; no weights to train
Max pooling	**CNN**	Keep the max of each block; 2×2 removes 75% of pixels
Preprocessing	**Training**	Mean subtraction → normalization → zero padding (after the train/test split)
Regularization	**Training**	Reduces overfitting; mechanism is weight decay (high weights fade toward zero)
L1 / L2 / Max-norm / Dropout / Early stopping	**Training**	The actual regularization *methods* (weight decay is the mechanism, not a method)
Recurrent layer	**RNN**	Feeds output into itself sequentially; can use only the last iteration's output
RNN types	**RNN**	One→many, many→one, many→many, and nonsequential input
LSTM	**RNN**	Adds a memory cell; gates learn what to forget, add, and output
Sigmoid (σ)	**LSTM**	Logistic unit; outputs 0–1, a percentage of what to keep
Tanh	**LSTM**	Unit that extracts information to add to, or read from, the cell
Language modeling	**RNN**	Assign probabilities to sentences by predicting the next word from history

CNN vs. RNN — at a glance

	CNN	RNN
Built for	Images & audio (spatial structure)	Sequences (order & time)
Key trick	Sliding filters + weight sharing	Feeding output back into itself
Memory	None needed — looks at space	Carries information across steps (LSTM adds long-term memory)
Typical tasks	Character recognition, image tagging, game play	Translation, sentiment analysis, video tagging, language modeling

★ Further Resources

Each section runs from conceptual introductions to hands-on implementation.

Convolutional Neural Networks (CNNs)

- **CS231n — Introduction to Convolutional Networks** — visual explanation of convolution, pooling, padding, and architecture.
  `https://cs231n.github.io/convolutional-networks/`
- **TensorFlow — CNN Tutorial** — builds a beginner CNN for image classification.
  `https://www.tensorflow.org/tutorials/images/cnn`
- **PyTorch — Training a CNN Classifier** — step-by-step image classifier using CIFAR-10.
  `https://docs.pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html`
- **PyTorch — Neural Networks Tutorial** — introduces convolutional layers, loss functions, and training.
  `https://docs.pytorch.org/tutorials/beginner/blitz/neural_networks_tutorial.html`
- **PyTorch — Transfer Learning for Computer Vision** — uses a pretrained CNN for a new task.
  `https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html`

Recurrent Neural Networks (RNNs)

- **Understanding LSTM Networks** — excellent visual introduction to RNNs, LSTMs, memory, and gates.
  `https://colah.github.io/posts/2015-08-Understanding-LSTMs/`
- **TensorFlow — Text Classification with an RNN** — builds an RNN for sentiment classification.
  `https://www.tensorflow.org/text/tutorials/text_classification_rnn`
- **TensorFlow — Text Generation with an RNN** — trains a character-level RNN to generate text.
  `https://www.tensorflow.org/text/tutorials/text_generation`
- **PyTorch — Character-Level RNN Classification** — builds an RNN from scratch to classify names.
  `https://docs.pytorch.org/tutorials/intermediate/char_rnn_classification_tutorial.html`
- **PyTorch — Sequence Models and LSTMs** — introduces sequence modeling and LSTM implementation.
  `https://docs.pytorch.org/tutorials/beginner/nlp/sequence_models_tutorial.html`
- **TensorFlow — Time-Series Forecasting** — RNN and LSTM models for sequential numerical data.
  `https://www.tensorflow.org/tutorials/structured_data/time_series`

> **SUGGESTED PATH** (1) Read the concept pages here → (2) CS231n for CNN intuition → (3) build the TensorFlow CNN on images → (4) read colah's LSTM post → (5) train an RNN for text, then time-series. Implement one small notebook after each step rather than reading everything first.